

Python Programming Assignment

Case Study: Automating Daily Sales Data Processing

Read the case study carefully before answering the questions in the worksheet.

Case Study: The Daily Sales Audit

Alex is a Operations Analyst at Apex Retail, a growing enterprise operating 12 regional branch stores. Every evening at 11:00 PM, each store generates a text-based comma-separated log file containing its daily sales records.

Currently, Alex spent an hour every morning opening all 12 text files manually, copying data into a master spreadsheet, stripping out malformed lines, filtering test transactions, and manually calculating revenue totals for each location.

Alex's team lead wants to automate this process using a custom Python script that reads raw text logs, cleans corrupted inputs, computes total revenue per store, and exports a clean summary log.

A Sample of Raw Transaction Logs

```
Log Entry #001 — "TXN1001, STORE_01, 45.50, CARD, SUCCESS"
Log Entry #002 — "TXN1002, STORE_02, -12.00, CASH, SUCCESS"    (Corrupted negative amount)
Log Entry #003 — "TXN1003, STORE_01, NONE, CARD, FAILED"      (Missing value string)
Log Entry #004 — "TEST_999, STORE_03, 0.00, CARD, SUCCESS"    (Test transaction)
Log Entry #005 — "TXN1004, STORE_02, 120.00, CASH, SUCCESS"
Log Entry #006 — "TXN1005, STORE_03, 89.99, CARD, PENDING"    (Unsettled payment status)
```

Alex notices specific structural rules across valid entries:

1. Every valid line contains 5 comma-separated values: `Transaction_ID`, `Store_ID`, `Amount`, `Payment_Type`, and `Status`.
2. Transaction strings read directly from `.txt` or `.csv` files are imported as Python strings (`str`) and contain whitespace and newline characters (`\n`).
3. Entries starting with `"TEST_"` or entries with status other than `"SUCCESS"` must be skipped during calculations.
4. Amounts must be converted into numerical float types, but invalid strings such as `"NONE"` raise runtime errors unless explicitly handled.

Key Metrics & Data Constraints

- **Daily Log Volume:** 12 regional log files containing roughly 1,500 total lines daily.
- **Error Frequency:** Approximately 4–6% of records contain malformed values or string formatting errors.
- **Target Python Environment:** Standard Python 3.x using built-in data structures and basic control flow.
- **Goal:** Write robust Python code that cleans, parses, and aggregates sales metrics without crashing on bad data.

Worksheet — Automated Sales Data Pipeline

Block 1: Fundamentals & Data Types

Question 1

When reading line-by-line from a text file in Python, each line is initially imported as a string (e.g., "45.50\n"). Which standard Python string method and type conversion sequence should Alex use to safely convert this into a usable number for arithmetic operations?

- ☐ (A) `int("45.50\n".replace())`
- ☐ (B) `float("45.50\n".strip())`
- ☐ (C) `str("45.50\n".split())`
- ☐ (D) `list("45.50\n".trim())`

Question 2

Alex needs to store an unordered collection of *unique* payment types encountered across all daily files (e.g., "CASH", "CARD"). Which standard Python data structure automatically prevents duplicate values?

- ☐ (A) List (`list`)
- ☐ (B) Tuple (`tuple`)
- ☐ (C) Set (`set`)
- ☐ (D) Dictionary (`dict`)

Question 3

To map each unique `Store_ID` to its running cumulative revenue total (e.g., {"STORE_01": 1500.50, "STORE_02": 890.00}), which Python data structure is most appropriate?

- ☐ (A) Dictionary (`dict`)
- ☐ (B) List of lists (`list`)
- ☐ (C) Tuple of strings (`tuple`)
- ☐ (D) Set (`set`)

Question 4

A line reads: `raw_line = "TXN1001, STORE_01, 45.50, CARD, SUCCESS"`. What will `raw_line.strip().split(",")` return in Python?

- ☐ (A) A single concatenated string without commas.
- ☐ (B) A dictionary with key-value pairs matching each field.
- ☐ (C) A list of 5 string elements: `['TXN1001', 'STORE_01', '45.50', 'CARD', 'SUCCESS']`.
- ☐ (D) A tuple containing 5 float numbers.

Block 2: Control Flow, Logic & Error Handling

Question 5

Alex wants to skip any transaction record where the `Transaction_ID` starts with "TEST_". Which Python conditional expression and string method correctly identifies these lines inside a loop?

- ☐ (A) `if txn_id.startswith("TEST_"): continue`
- ☐ (B) `if txn_id.find("TEST_") == True: break`
- ☐ (C) `if txn_id.islower("TEST_"): pass`
- ☐ (D) `if txn_id == "TEST_": exit()`

Question 6

When processing Log Entry #003 (`Amount = "NONE"`), attempting `float("NONE")` raises a `ValueError` and crashes the program. How should Alex structure the code to safely skip malformed numbers?

- ☐ (A) Use an `if...else` block checking if `Amount == float`.
- ☐ (B) Wrap the conversion in a `try...except ValueError:` block to catch and handle the error gracefully.
- ☐ (C) Use the `sys.exit()` function whenever a non-digit character is detected.
- ☐ (D) Delete the text file containing the error line before running the script.

Question 7

Which of the following file handling practices ensures that log files are automatically closed properly, even if a runtime error occurs during reading?

- ☐ (A) `file = open("sales.txt", "r")`
- ☐ (B) `with open("sales.txt", "r") as file:`
- ☐ (C) `for line in open("sales.txt").close():`
- ☐ (D) `import file; file.read("sales.txt")`

Question 8 — Code Construction

Write a short, complete Python function named `clean_and_parse` that takes a raw log string (e.g., "TXN1001, STORE_01, 45.50, CARD, SUCCESS\n") as input and returns a tuple of (`store_id`, `amount`) if the record is valid, or `None` if invalid.

Validation rules: Must have status "SUCCESS", amount > 0, and non-test ID.

Block 3: Practical Implementation & Analysis

Question 9 — Written

In 2–3 sentences, explain why using explicit Exception Handling (`try...except`) is superior to pre-checking every possible string error using nested `if...else` statements when processing large, unpredictable input data files.

Question 10 — Written

In Log Entry #003, the amount is listed as "NONE", while in Log Entry #004, a test transaction amount is listed as 0.00. Describe how Alex's script should distinguish between a *missing value error* versus a *valid zero-dollar transaction*, and why they must be handled differently.

Question 11 — Written

Suppose the business expands from 12 regional stores to 500 branch locations, and daily log processing needs to scale. What modular adjustments (e.g., functions, data structures, external modules) should Alex make to keep the Python codebase maintainable and reusable?

Question 12 — Written

Propose two automated validation checks that Alex's script should run at the very end of processing before exporting the summary report to alert management of potential data corruption across regional branch files.
